Metro Ticket Generating System (ServiceNow)

Project Documentation

1. Introduction

- **Project Title:** Metro Ticket Generating System (built on ServiceNow)
- **Team Members:**
  - Team Member 1 – Project lead and ServiceNow developer (catalog item, variables, client scripts)
  - Team Member 2 – Data architecture and automation (tables, Flow Designer)
  - Team Member 3 – Testing and security (UAT, ACLs, defect fixing)
  - Team Member 4 – QR widget, documentation and demo

2. Project Overview

- **Purpose:** Digitise metro ticket booking so that passengers can book online, see the fare instantly and get a QR code ticket, while every booking is stored for station staff. This reduces queues, paper usage and manual work.
- **Features:**
  - 'Book A Metro Ticket' Service Catalog item on the Service Portal
  - Station selection from the Metro Station's Details table
  - Single / Return journey and 1–4 passengers
  - Automatic fare calculation (catalog client script)
  - Payment mode UPI / Card / Others with UI policy
  - QR code ticket in a portal widget popup
  - Request (REQ/RITM) tracking in My Requests
  - Flow Designer stores each booking and creates a task for Station Managers
  - Default ACLs protect the custom tables

3. Architecture

- **Frontend:** ServiceNow Service Portal (/sp). The catalog item renders the booking form; catalog client scripts (JavaScript, g_form API) calculate the fare and generate the QR; a catalog UI policy controls field visibility; the metro_qr_widget (HTML template + server script, AngularJS) shows the QR popup through spModal.

• **Backend:** ServiceNow platform. The Service Catalog creates the request (REQ) and requested item (RITM). 'Metro Ticket Booking Flow' (Flow Designer, run as System) is triggered by the catalog item and runs Get Catalog Variables → Create Record → Create Catalog Task.

• **Database:** ServiceNow tables. u_metro_station_details (u_station_name) stores stations; u_metro_ticket_booking (auto number MTB) stores bookings with references to stations, user and requested item; platform tables sc_request, sc_req_item, sc_task and sys_user hold requests, tasks and users.

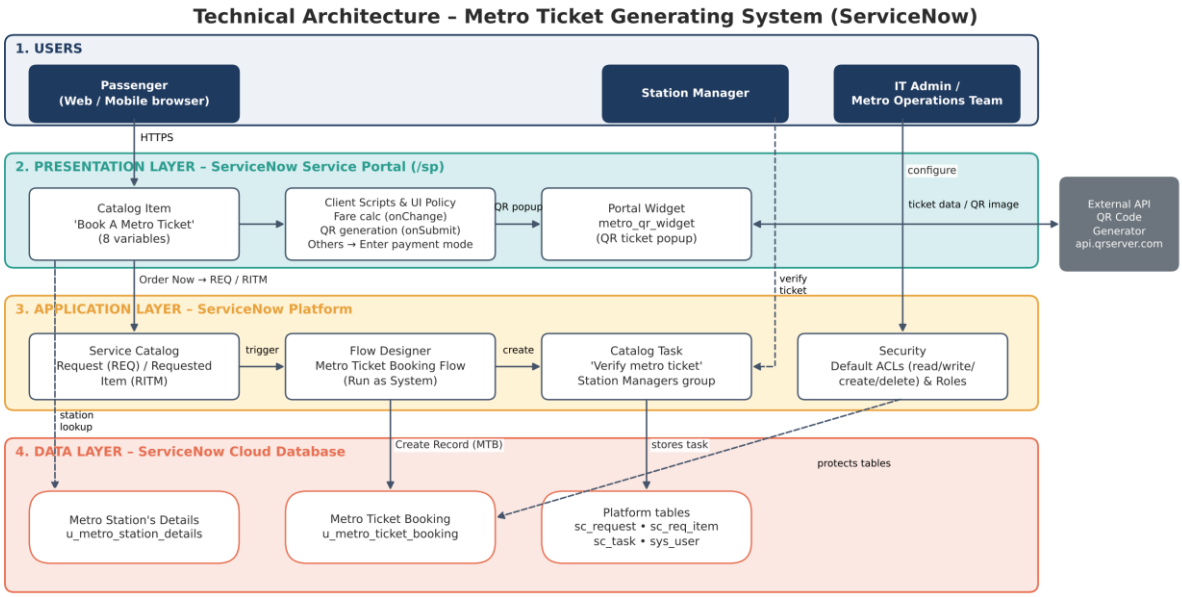


Figure 1: Technical architecture

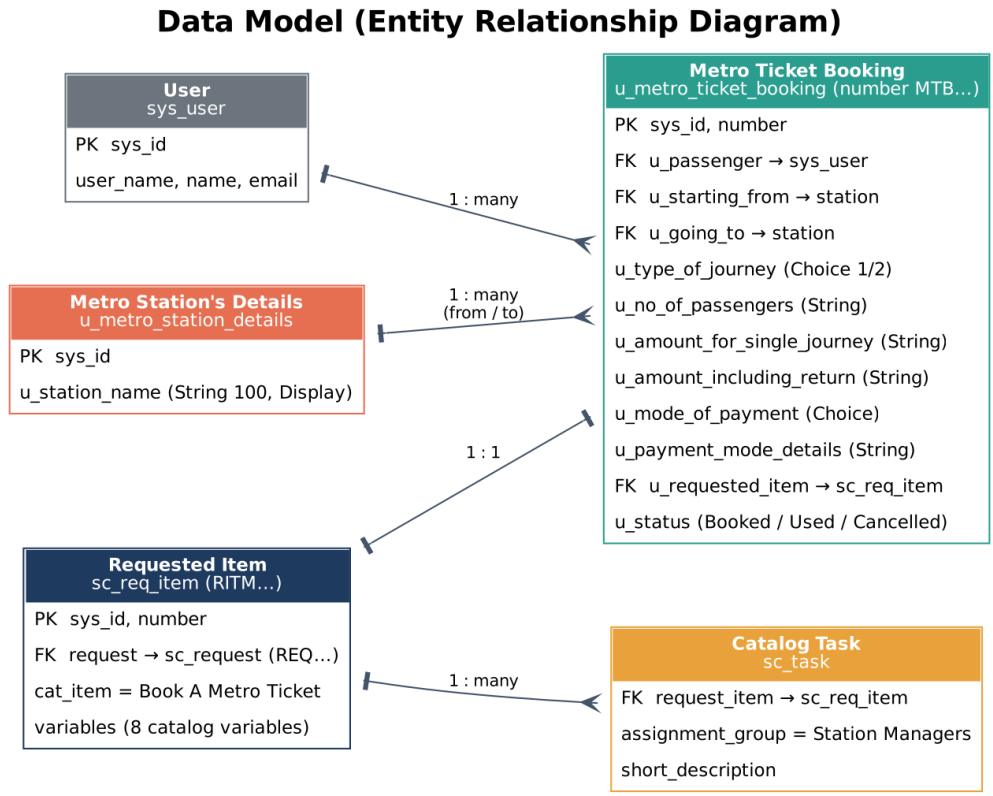


Figure 2: Data model (entity relationship diagram)

4. Setup Instructions

• **Prerequisites:** ServiceNow developer account, a Personal Developer Instance (Zurich or Australia release), a modern web browser (Chrome). No Node.js or MongoDB is needed.

• **Installation (configuration) steps:**

1. Log in to the PDI as admin and create the update set 'Metro Ticket Booking' (Global scope).

2. Create table Metro Station's Details with field Station Name (set as Display) and add 5 stations.

3. Create catalog item 'Book A Metro Ticket' (Service Catalog → Services) with 8 variables and their choices.

4. Add the onChange fare client script on No of Passengers using the instance's own station sys_ids.

5. Add the catalog UI policy: Mode of Payment is Others → Enter payment mode visible and mandatory.

6. Create widget 'Metro QR Widget' (ID metro_qr_widget) and the onSubmit QR client script.

7. Create table Metro Ticket Booking, the Station Managers group and 'Metro Ticket Booking Flow'; attach the flow in the item's Process Engine tab.

8. Test on /sp and confirm the REQ number, MTB record, catalog task and flow execution.

5. Folder Structure (ServiceNow application components)

• **Client side (Service Portal)**: Catalog item 'Book A Metro Ticket' → Variables (8) → Catalog Client Scripts (Auto populate Fare - Passengers, Generate QR Code) → Catalog UI Policy (Others) → Widget metro_qr_widget.

• **Server side (platform):** Tables u_metro_station_details and u_metro_ticket_booking → Flow 'Metro Ticket Booking Flow' → Group 'Station Managers' → Default ACLs → Update set 'Metro Ticket Booking'.

6. Running the Application

• Open https://<instance>.service-now.com/sp and log in.

• Search 'Book A Metro Ticket', choose Starting From, Going To, Type of journey and No of Passengers – the amount fills automatically.

• Choose Mode of Payment, click Order Now → Checkout to see the QR ticket, click OK, then Order Now → Checkout again to place the order and get the REQ number.

• **Admin view:** open Metro Ticket Booking list, Requested Items and Flow executions (sys_flow_context.list) to see the stored booking, task and flow status.

7. API Documentation

• No custom REST endpoints are exposed by this application. Bookings are created through the Service Catalog.

• External API used – QR Code Generator:

o **Method:** GET

o **Endpoint:** https://api.qrserver.com/v1/create-qr-code/

o **Parameters:** size=250x250, data=<URL-encoded ticket text>

o **Example:** https://api.qrserver.com/v1/create-qr-code/?size=250x250&data=https%3A%2F%2Fexample.com%2Fticket%3Fid%3D<sys_id>

o **Response:** PNG image of the QR code, displayed in the metro_qr_widget popup.

8. Authentication

• Users log in with ServiceNow authentication; the Service Portal requires a logged-in user and the booking is requested for that user.

• Authorization uses the default ACLs (read, write, create, delete) created with each custom table – only users with the table role or admins can open the tables.

• The flow runs as System, so bookings are stored even though passengers cannot open the table. This was verified by impersonating a test user with no roles.

9. User Interface

• Booking form: 8 fields with mandatory marks; amount fields are read-only; Enter payment mode appears only for Others.

• **QR popup:** 'Your Metro QR Ticket' with a 250×250 QR image and the text 'Scan this QR code at the metro station gate'.

• Order confirmation and order status page with the REQ number; Requested Item page with the booking details.

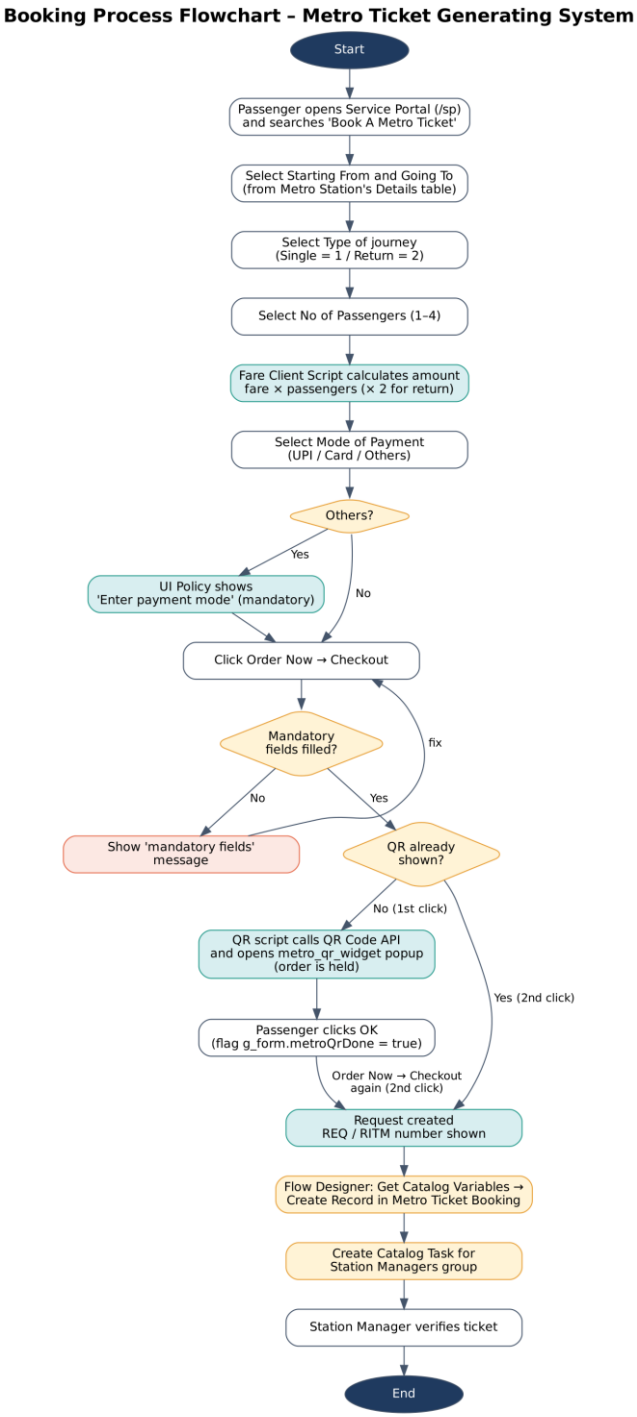
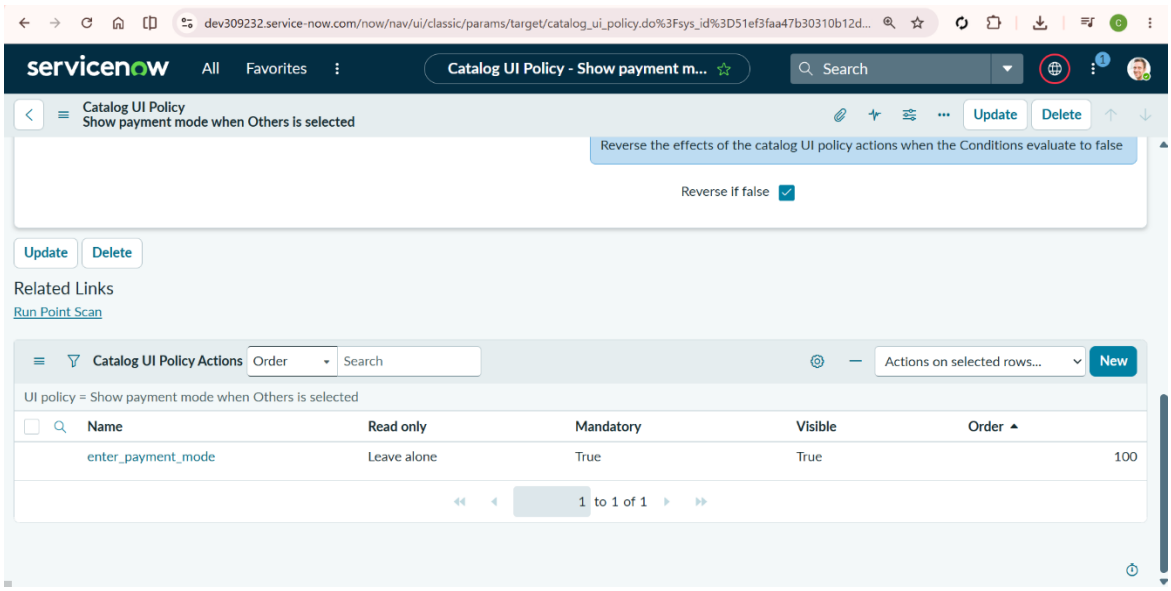
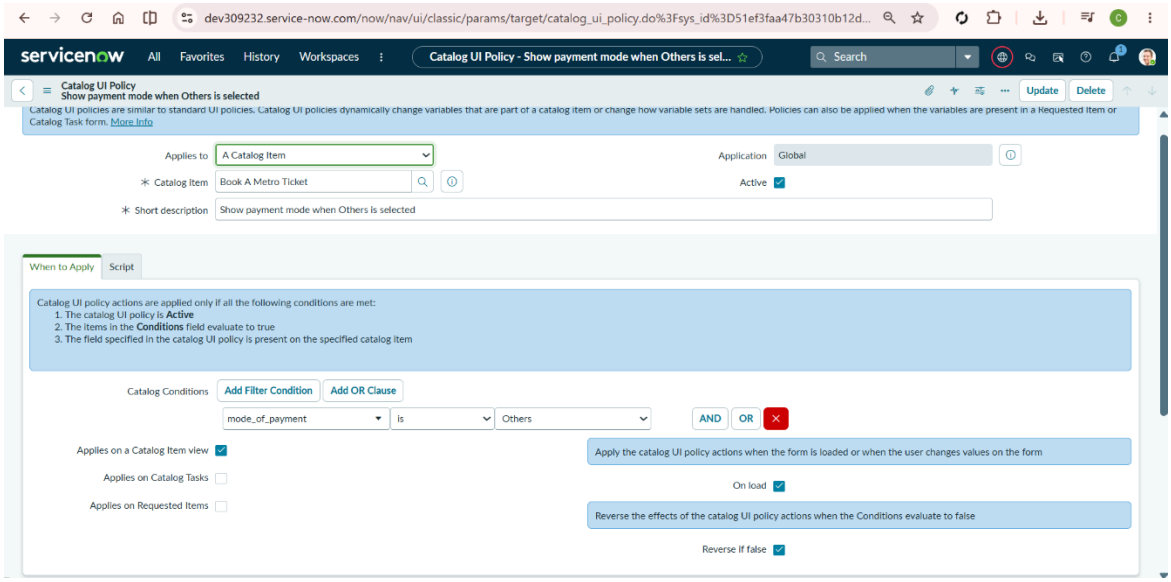


Figure 3: Booking process flowchart



UI policy action — show Enter payment mode.



UI policy — condition Mode of Payment is Others.

10. Testing

• **Strategy:** functional testing of every feature, followed by User Acceptance Testing on the Service Portal with positive and negative cases.

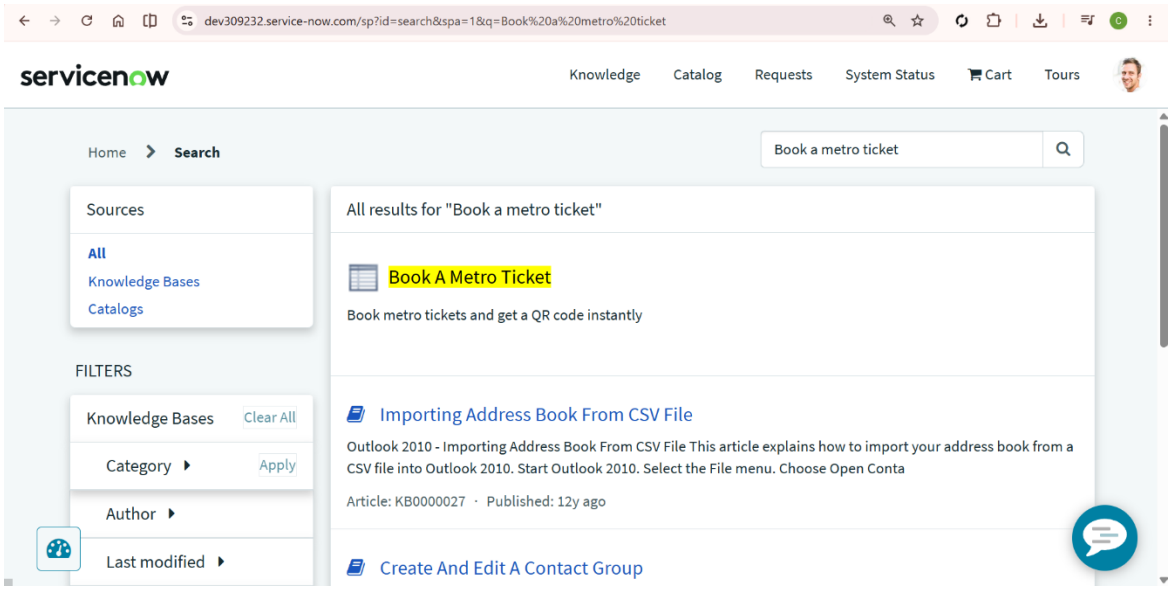
• **Result:** 15 UAT test cases executed – 15 passed. 8 defects logged – 6 closed, 2 open (known limitations).

• **Tools:** ServiceNow Service Portal, Impersonate User, Flow execution details, browser console (F12) for client-script errors.

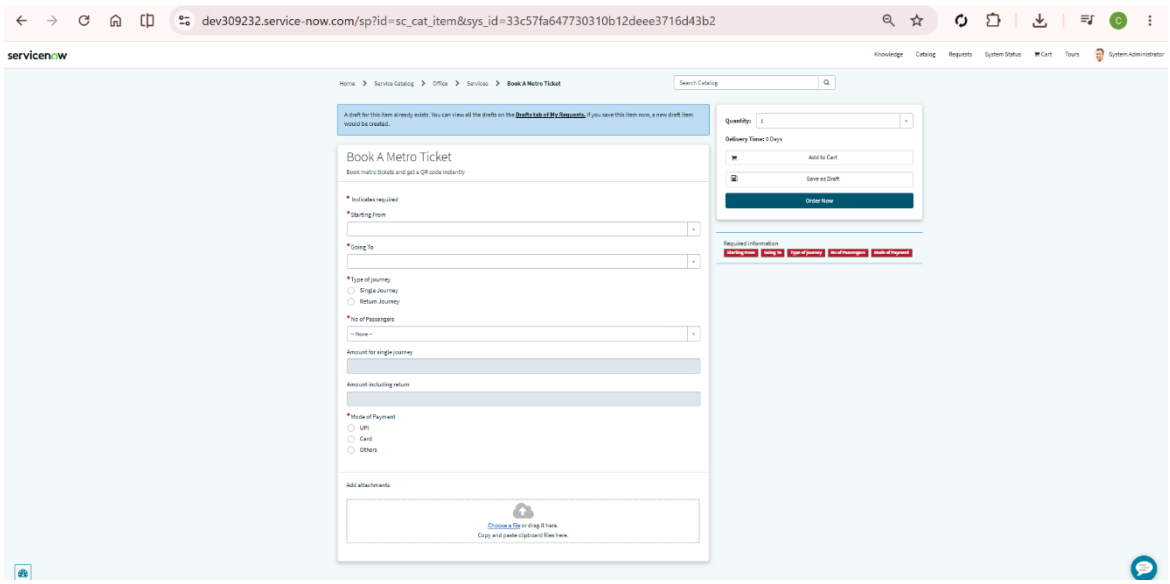
11. Screenshots or Demo

• **Demo video link:**
https://drive.google.com/file/d/1AtEF1eX5mBMuy5Oe_T1a0HMlIhbBOS1s/view?usp=sharing

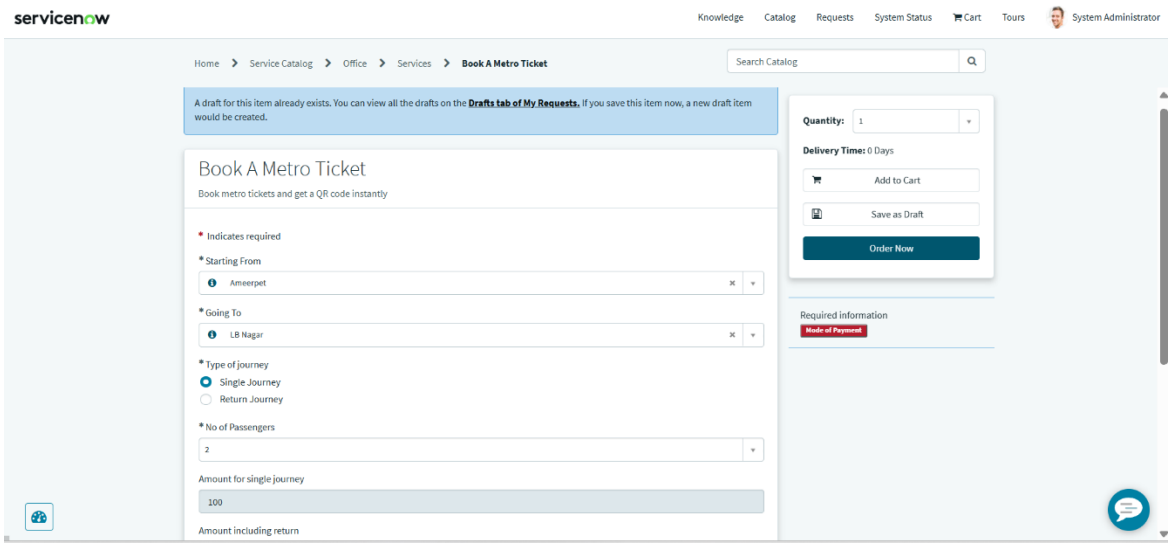
· Screenshots of the flow:



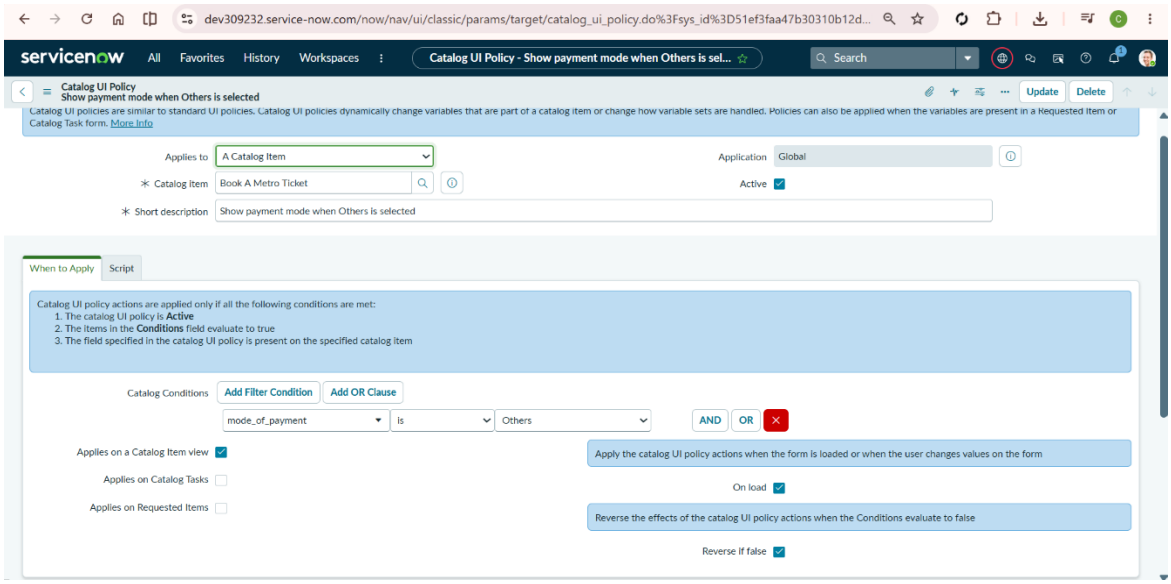
Book A Metro Ticket on the Service Portal



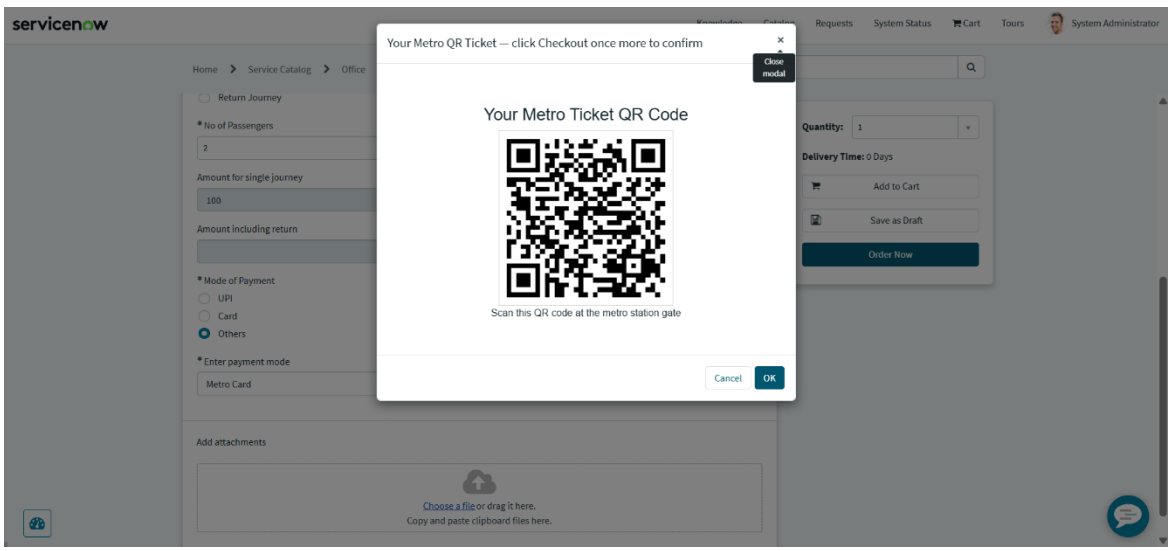
Metro ticket booking form.



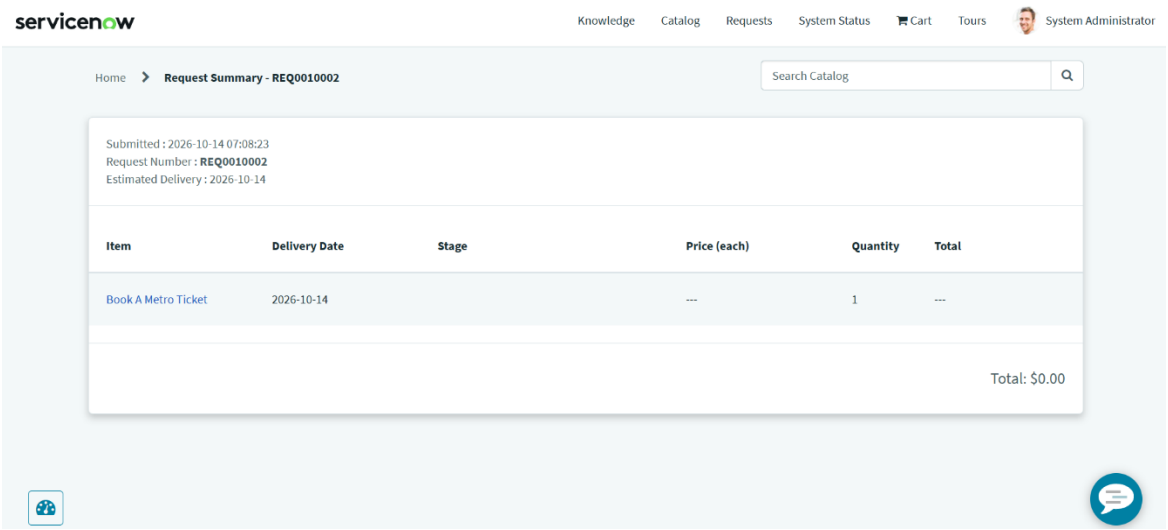
Fare calculated automatically (₹50 × 2 passengers = ₹100).



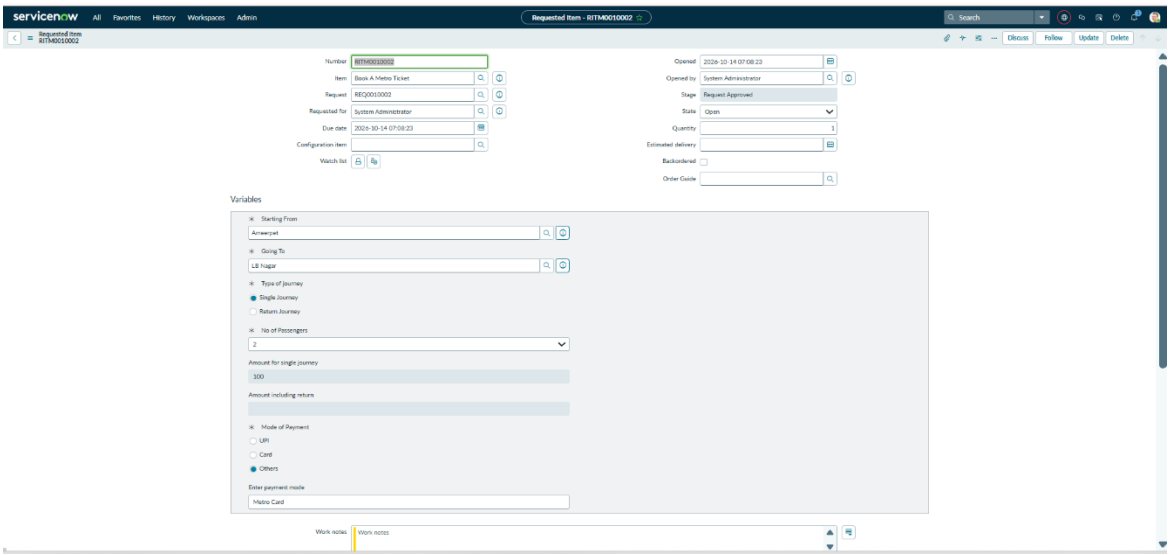
Others UI policy



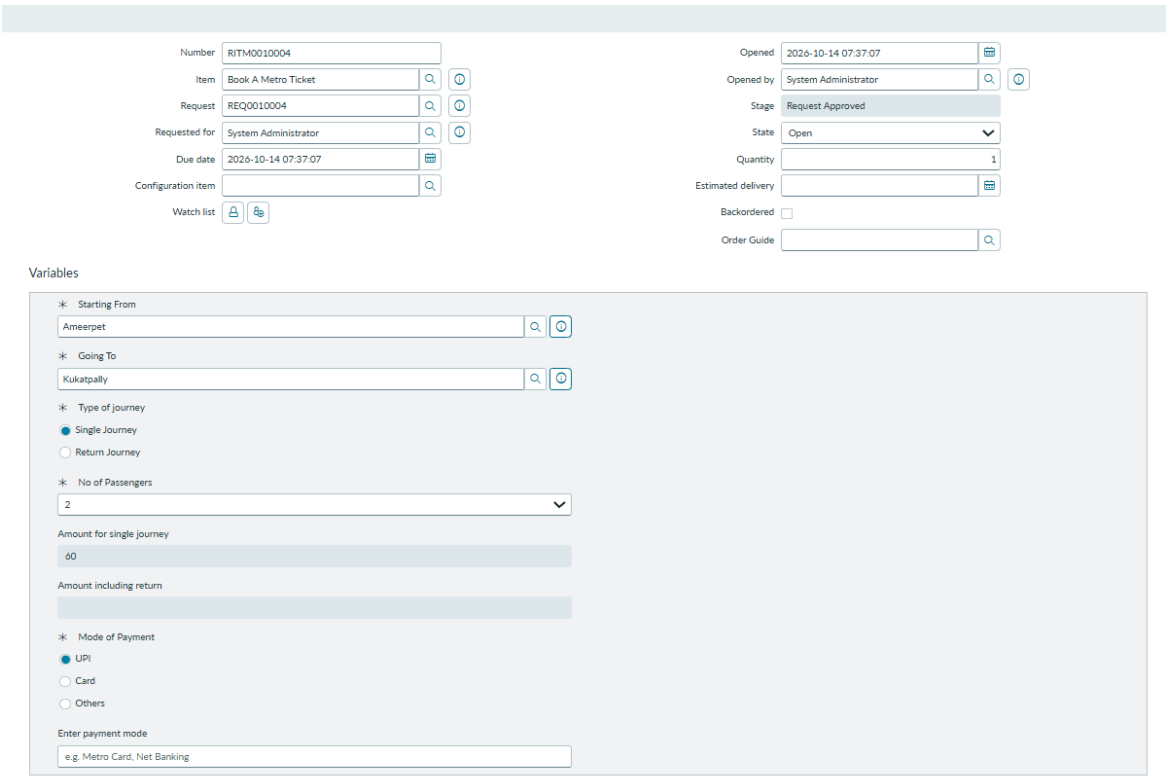
QR ticket generated



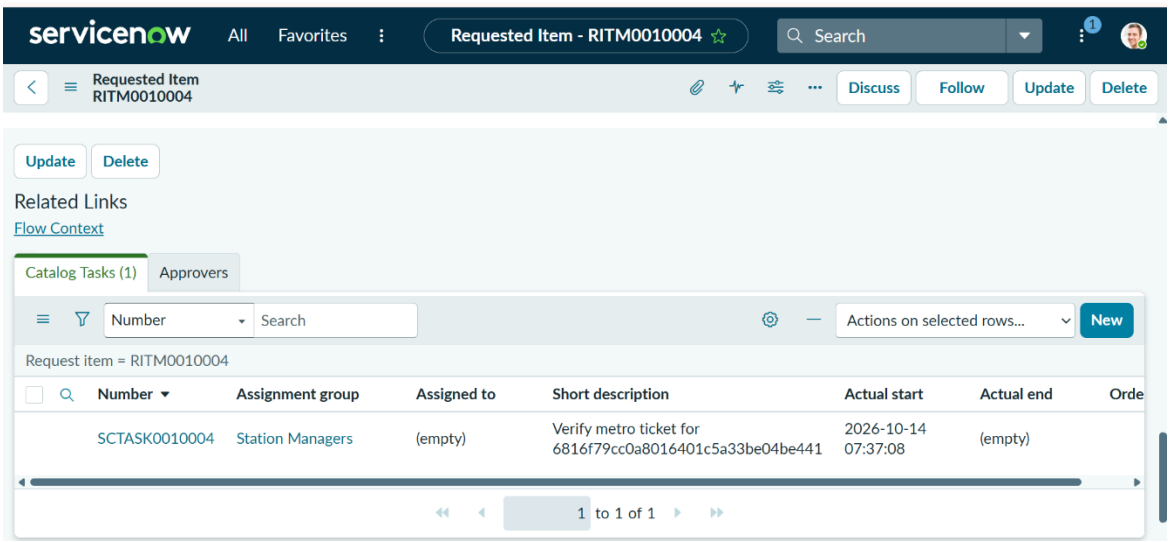
Request submitted with request number.



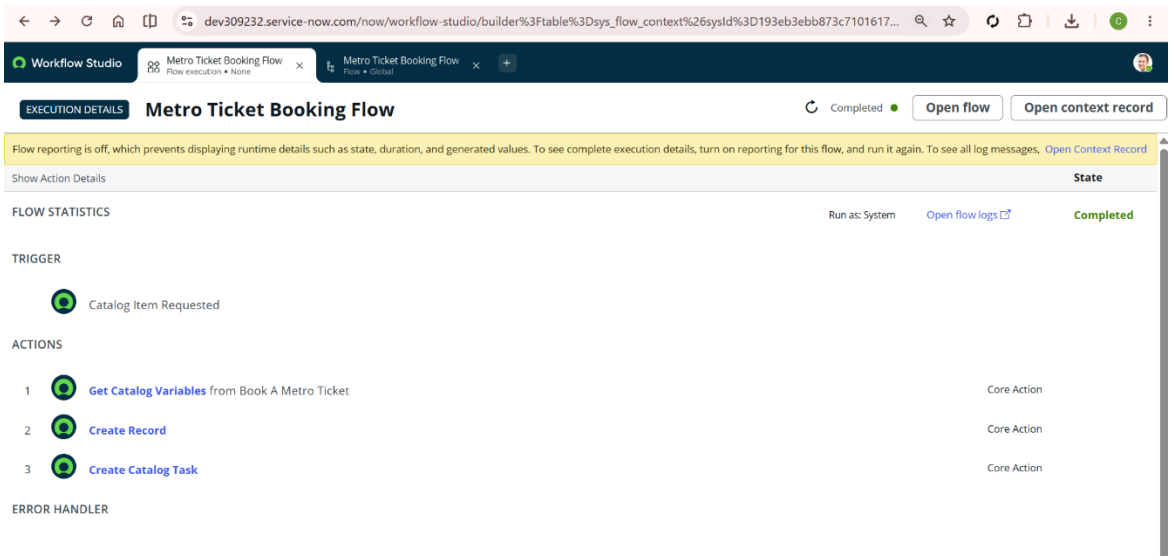
Requested Item page with booking details.



Booking data stored in Metro Ticket Booking table (via Flow Designer).



Verification task created for Station Managers.



Flow execution completed successfully.

12. Known Issues

- Fare updates only when No of Passengers changes – select passengers last.
- Fares are configured only for routes from Ameerpet; other routes leave the amount empty.
- Order Now must be clicked twice (first click shows the QR ticket).
- Both amount fields are visible before a journey type is chosen (kept as per the project guide).
- The QR code contains a ticket link, not full ticket details; no real payment is taken.

13. Future Enhancements

- Payment gateway integration (UPI / cards / wallets).
- SMS, email and WhatsApp ticket delivery.
- Automated Fare Collection gates that scan the QR code.
- Fares for every station pair and multiple metro lines.
- Travel analytics dashboard: peak hours, busiest routes, payment-mode share.